МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное автономное образовательное учреждение высшего образования «Санкт-Петербургский политехнический университет Петра Великого»

Институт компьютерных наук и кибербезопасности

Высшая школа технологий и искусственного интеллекта

Направление: 02.03.01 Математика и компьютерные науки

Отчет по курсовой работе по дисциплине "Функциональное программирование" Реализация проекта на языке Haskell по теме "Визуализатор маршрутов"

Отчет выполнил
студент группы 5130201/40001 _____ Костина А.В.
Отчет принял
преподаватель _____ Моторин Д.Е.

«____»_____ 2026г.

Санкт-Петербург, 2026

Содержание

Техническое задание **3**

Введение **11**

1 Постановка задач **12**

2 Особенности реализации **13**
2.1 Описание разработки проекта 13
2.1.1 Структура проекта 13
2.1.2 Диаграмма модулей 13
2.2 Описание технических особенностей реализации проекта 15

3 Примеры работы программы **16**

4 Руководство по эксплуатации для пользователя **17**

Заключение **18**

Список литературы **19**

Приложение **20**
A.1 20

Техническое задание

1. Основание для выполнения работы

Работа выполняется в соответствии с учебным планом по дисциплине «Функциональное программирование» в рамках курсового проектирования студента группы 5130201/40001 Института компьютерных наук и кибербезопасности Федерального государственного автономного образовательного учреждения высшего образования «Санкт-Петербургский политехнический университет Петра Великого», обучающегося по направлению подготовки 02.03.01 Математика и компьютерные науки.

2. Исполнитель и соисполнители работы

Исполнитель: студент группы 5130201/40001
Соисполнители: отсутствуют

3. Цели, задачи и исходные данные

a. Цель работы

Целью работы является разработка наглядного инструмента для поиска и визуализации кратчайших маршрутов в транспортных сетях (метрополитен, железные дороги).

b. Задачи работы

В процессе выполнения работы требуется решить следующие задачи:

- **Чтение и запись файлов:**
  - реализовать импорт графа из JSON-файла, описывающего схему метро/ЖД с использованием библиотеки `aeson`;
  - реализовать экспорт графа в JSON-файл;
  - реализовать сохранение визуализаций графа и найденного кратчайшего пути в PNG-файлы.
- **Обработка информации математическими функциями:**
  - разработать структуры данных для представления взвешенных графов (ориентированных/неориентированных);

  - реализовать алгоритм Дейкстры для поиска кратчайшего пути;
  - реализовать восстановление маршрута от начальной до конечной вершины.

- **Взаимодействие с пользователем через консоль:**
  - разработать систему меню для управления программой;
  - обеспечить ввод начальной и конечной станций для построения маршрута.

- **Логирование действий пользователя:**
  - реализовать запись в лог-файл всех значимых действий;
  - добавить временные метки и уровни логирования.

- **Обработка ошибок:**
  - обеспечить корректную обработку ошибок ввода/вывода;
  - обрабатывать некорректный пользовательский ввод;
  - обрабатывать ситуации, когда путь между станциями отсутствует.

c. Исходные данные

Исходными данными для проведения работы являются:

- JSON-файлы с описанием графов транспортных сетей в формате, включающем названия станций и перегоны с весами;
- готовые датасеты транспортных сетей (например, из открытых источников);

4. Основное содержание работы

a. Разработка проекта

Разработка проекта включает:

- проектирование архитектуры и системы типов;
- реализацию модулей для работы с графами, алгоритмом Дейкстры, визуализацией, вводом/выводом JSON, логированием и обработкой ошибок;
- написание тестов с использованием QuickCheck;
- отладку и тестирование программы.

b. Отчетная документация

Отчёт и программа на языке Haskell.

5. Основные требования к выполнению работы

a. Назначение разрабатываемого программного комплекса

Программный комплекс предназначен для поиска и визуализации кратчайших маршрутов в транспортных сетях (метро, ЖД). В зависимости от выбора пользователя оценивает время прохождения или длину маршрута по транспортным сетям. Может использоваться как вспомогательный инструмент для планирования поездок.

b. Состав разрабатываемого программного комплекса

- Система сборки: Stack;
- JSON-файлы с примерами схем метро и ЖД;
- документация пользователя.

c. Технические характеристики

Файлы, используемые программой:

- JSON-файл с описанием транспортной сети (станция, линия, координаты); Например:

```
{
 "lines": [
 {
  "hex_color": "CD0505",
  "stations": [
  {
   "lat": 53.20969,
   "lng": 50.13442,
   "order": 0,
   "name": "Алабинская",
```

```
 "admArea": null,
 "district": null,
 "status": null
},
{
 "lat": 53.2113838,
 "lng": 50.15022,
 "order": 1,
 "name": "Российская",
 "admArea": null,
 "district": null,
 "status": null
},
{
 "lat": 53.2037926,
 "lng": 50.159832,
 "order": 2,
 "name": "Московская",
 "admArea": null,
 "district": null,
 "status": null
},
{
 "lat": 53.2004433,
 "lng": 50.1765938,
 "order": 3,
 "name": "Гагаринская",
 "admArea": null,
 "district": null,
 "status": null
},
{
 "lat": 53.2011223,
 "lng": 50.1992874,
 "order": 4,
 "name": "Спортивная",
 "admArea": null,
 "district": null,
 "status": null
},
{
 "lat": 53.2017365,
 "lng": 50.2206573,
```

```
 "order": 5,
 "name": "Советская",
 "admArea": null,
 "district": null,
 "status": null
},
{
 "lat": 53.2073135,
 "lng": 50.2364159,
 "order": 6,
 "name": "Победа",
 "admArea": null,
 "district": null,
 "status": null
},
{
 "lat": 53.2129974,
 "lng": 50.24889,
 "order": 7,
 "name": "Безымянка",
 "admArea": null,
 "district": null,
 "status": null
},
{
 "lat": 53.2113533,
 "lng": 50.2697754,
 "order": 8,
 "name": "Кировская",
 "admArea": null,
 "district": null,
 "status": null
},
{
 "lat": 53.2127,
 "lng": 50.28298,
 "order": 9,
 "name": "Юнгородок",
 "admArea": null,
 "district": null,
 "status": null
}
],
```

```
    "id": "54",
    "name": "Первая"
   }
   ],
   "name": "Самара"
  }
```

Исходные данные получены из репозитория: `https://github.com/morphey83/MetroAndDistrict/blob/master/metro.all.json` (Дата обращения 03.04.2026)

- JSON-файл для экспорта текущей схемы;
- PNG-файл визуализации всей схемы;
- PNG-файл визуализации найденного маршрута;
- текстовый файл лога с временными метками и уровнями логирования.

Что технически может сделать пользователь:

- Загрузить схему из JSON-файла;
- Экспортировать текущую схему в JSON-файл;
- Просмотреть список всех станций;
- Найти станцию по части названия;
- Выбрать начальную и конечную станцию;
- Запустить алгоритм Дейкстры для поиска кратчайшего пути;
- Просмотреть маршрут и его длину;
- Сохранить визуализацию всей схемы в PNG;
- Сохранить визуализацию маршрута с выделенным путём в PNG;
- Просмотреть последние записи лога или весь лог;
- Очистить лог-файл.

d. Функциональные требования

d.1. Требования к производительности

- Загрузка и парсинг JSON-файла — не более 2 секунд.
- Работа алгоритма Дейкстры для графа до 500 вершин — не более 0.5 секунды.

- Генерация PNG-визуализации — не более 3 секунд.
- Потребление оперативной памяти — не более 200 МБ.

d.2. Требования к точности и обработке данных

- Алгоритм Дейкстры должен находить кратчайший путь с точностью 100%.
- Веса рёбер поддерживаются дробными (точность до 0.1).
- При импорте JSON выполняется проверка ссылочной целостности.
- Программа корректно обрабатывает неориентированные графы.

d.3. Требования к хранению и логированию

- Все действия пользователя фиксируются в лог-файле.
- Максимальный размер лога — 10 МБ, при превышении старая запись перезаписывается.
- Экспорт сохраняет полную структуру графа без потерь.

d.4. Требования к надёжности

- Программа не должна завершаться аварийно при некорректном вводе.
- При отсутствии пути между станциями выводится сообщение об ошибке.
- Визуализация невозможна без загруженной схемы.

e. Стандартные средства

При разработке должны использоваться типовые средства языка Haskell и существующие библиотечные решения:

- `aeson` — для парсинга и генерации JSON-файлов;
- `containers` (Data.Map, Data.Set, Data.Sequence) — для представления графов и структур данных алгоритмов;
- `graphite` — для визуализации графов и сохранения в PNG;
- `QuickCheck` — для тестирования свойств;
- `directory` — для работы с файловой системой;
- `time` — для формирования временных меток в логах;
- `bytestring` — для эффективной работы с JSON.

6. Этапы и сроки выполнения работы

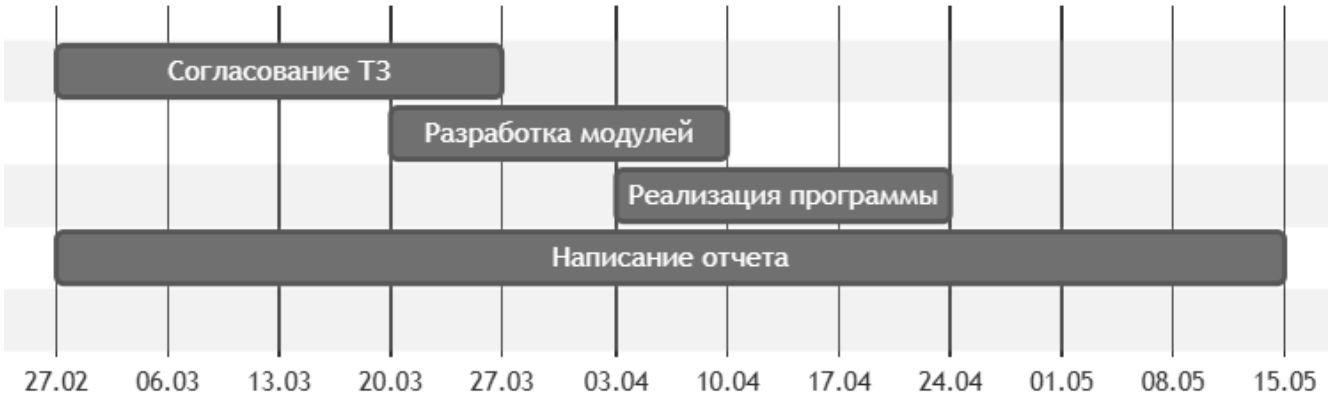


Рис. 1: Диаграмма Ганта этапов работ

7. Результаты работы

1. Утверждённое техническое задание.
2. Диаграмма модулей, описание типов данных, обоснование выбора библиотек.
3. Работающие модули парсинга JSON, поиска маршрута (алгоритм Дейкстры), визуализации, консольного интерфейса, логирования и обработки ошибок.
4. Готовый отчёт с описанием разработки, примерами работы и руководством пользователя.

Введение

Сейчас транспортные системы становятся всё сложнее: метро, железные дороги и их пересечения образуют разветвлённые сети. Поэтому умение быстро находить оптимальный маршрут от точки А до точки Б важно не только для разработчиков навигаторов, но и для многих IT-специалистов, работающих с данными, визуализацией или логистикой.

Для моделирования таких сетей удобно использовать графы [1]. В этой модели станции становятся вершинами, перегоны между ними — рёбрами, а веса рёбер могут означать время поездки или расстояние. Классическим алгоритмом поиска кратчайшего пути во взвешенном графе является алгоритм Дейкстры [2]. Понимание того, как он работает и какие структуры данных (например, очереди с приоритетом) нужны для его эффективной реализации, помогает правильно решать задачу маршрутизации.

Помимо самого поиска пути, важна практическая работа с данными. Схемы транспортных сетей удобно хранить в формате JSON [3], а для их парсинга в Haskell использовать библиотеку `aeson` [4]. Также важна визуализация: найденный маршрут должен быть не просто списком станций, но и наглядной картинкой. Для этого подходит библиотека `graphite` [5], которая позволяет строить схему по координатам станций и сохранять результат в PNG.

1 Постановка задач

2 Особенности реализации

2.1 Описание разработки проекта

2.1.1 Структура проекта

```
route-visualizer/
  app/
    Main.hs               # Точка входа
  src/
    CLI/
      Menu.hs             # Консольный интерфейс
    Graph/
      Types.hs            # Типы данных
      Loader.hs           # Загрузка/экспорт JSON
      Dijkstra.hs         # Алгоритм Дейкстры
      Utils.hs            # Вспомогательные функции
    Visualization/
      Render.hs           # Визуализация в PNG
    Logging/
      Logger.hs           # Логирование
  test/
    Spec.hs               # Тесты QuickCheck
  data/
    metro.json            # Входные данные
  logs/
    app.log               # Лог-файл
  output/
    full_graph.png
    route.png
  package.yaml            # Stack конфигурация
  stack.yaml
```

2.1.2 Диаграмма модулей

Main.hs

main:: IO ()
точка входа в программу
запускает главное меню
логирует старт и завершение

CLI/Menu.hs

runCLI :: IO ()
главный цикл меню
отображает 10 пунктов
обрабатывает выбор пользователя

String

String

String

String

getLine :: String

putStrLn :: String

String

String

String

String

String

loadGraphHandler
запрашивает путь к JSON
вызывает loadGraphFromJSON
сохраняет Graph в состояние

exportGraphHandler
запрашивает путь к JSON
вызывает exportGraphToJSON
экспортирует текущий граф

visualizeGraphHandler
запрашивает путь к PNG
вызывает renderGraphToPNG
сохраняет визуализацию

visualizePathHandler
запрашивает путь к PNG
вызывает renderPathToPNG
сохраняет путь на карте

findPathHandler
запрашивает start и target
вызывает shortestPath
отображает маршрут и длину

findStationHandler
запрашивает часть названия
ищет станции по подстроке
выводит найденные станции

listStationsHandler
выводит все станции графа
сортирует по названию
показывает линии

viewLogHandler
вызывает readLog
отображает последние записи
показывает timestamp и уровень

clearLogHandler
подтверждает действие
вызывает clearLog
очищает файл лога

Graph/Loader.hs

Either String Graph
Left errMsg :: String
Right graph :: Graph

filePath :: FilePath

loadGraphFromJSON
читает JSON файл
парсит в JsonMetro
преобразует в Graph

graph :: Graph
filePath :: FilePath

JsonMetro

Graph

buildGraph
JsonMetro → Graph
строит Map станций
строит список смежности

exportGraphToJSON
преобразует Graph в JSON
сериализует
записывает в файл

Vizualization/Render.hs

graph :: Graph
filePath :: FilePath

graph :: Graph
path :: PathResult
filePath :: FilePath

renderGraphToPNG
конвертирует Graph в Dot
вызывает dot -Tpng
сохраняет PNG файл

renderPathToPNG
подсвечивает путь на графе
конвертирует в Dot
сохраняет PNG файл

Graph

DotGraph Node

(Graph, Set StationId)

DotGraph Node

toDot
Graph → DotGraph Node
создаёт вершины и рёбра
задаёт стили

toDotWithHighlight
Graph → Set StationId → DotGraph
подсвечивает вершины пути
меняет цвет

Graph/Dijkstra.hs

graph :: Graph
start :: StationId
target :: StationId

PathResult

shortestPath
алгоритм Дейкстры
находит кратчайший путь
возвращает PathResult

(Graph, StationId)

(Graph, start, target, DistanceMap)

[StationId]

Map StationId Weight

dijkstra
внутренняя функция
вычисляет таблицу расстояний
использует State монаду

reconstructPath
восстанавливает маршрут
по таблице расстояний
возвращает список станций

Logging/Logger.hs

()

[LogEntry]

()

readLog
читает app.log
возвращает [LogEntry]
IO [LogEntry]

logMessage
LogLevel → Text → IO ()
добавляет timestamp
записывает в app.log

clearLog
очищает app.log
подтверждает действие
IO ()

()

checkLogSize
проверяет размер файла
если >10 MB вызывает rotateLog
IO ()

()

rotateLog
переименовывает app.log
создаёт новый пустой файл
IO ()

Внешние данные

FilePath -> ByteString

ByteString -> FilePath

metro.json
JSON файл со схемой метро

graph.png
PNG визуализация всего графа

path.png
PNG визуализация с путём

Консоль
ввод/вывод пользователя

app.log
текстовый файл лога

Graph/Types.hs

Graph
stations :: Map StationId Station
edges :: Map StationId [(StationId, Weight)]
isDirected :: Bool

PathResult
NoPath
Path { nodes :: [StationId], weight :: Weight }

StationId :: Text

Weight :: Double

LogLevel
Debug | Info | Warning | Error

LogEntry
timestamp :: UTCTime
level :: LogLevel
message :: Text

truncate app.log

app.log -> [LogEntry]

[LogEntry] -> app.log

2.2 Описание технических особенностей реализации проекта

3 Примеры работы программы

4 Руководство по эксплуатации для пользователя

Заключение

Список литературы

[1] Новиков Ф.А. Дискретная математика для программистов. 3-е изд. – Санкт-Петербург : Питер Пресс, 2009. – 384 с.

[2] Левитин А. В. Алгоритмы: введение в разработку и анализ. – М. : Вильямс, 2006. – 576 с.

[3] RFC 8259: The JavaScript Object Notation (JSON) Data Interchange Format. – URL: `https://datatracker.ietf.org/doc/html/rfc8259` (дата обращения: 09.04.2026).

[4] Hackage: aeson – A JSON parsing and encoding library for Haskell. – URL: `https://hackage.haskell.org/package/aeson` (дата обращения: 09.04.2026).

[5] Hackage: graphite – A library for rendering graphs and exporting to PNG. – URL: `https://hackage.haskell.org/package/graphite` (дата обращения: 09.04.2026).

Приложение

A.1